



TECNOLÓGICO NACIONAL DE MÉXICO

INSTITUTO TECNOLÓGICO DE AGUASCALIENTES

Tecnologías Inalámbricas

Unidad 4

Dávila Jasso Axhel Giovanni

23150315

Flores Dueñas Elvia Yuridia

23151302

Evaluativo FINAL

Semestre 7

Ricardo Alejandro Rodríguez Jiménez

05 de junio de 2026

Índice

Tabla de contenido

Introducción	3
Desarrollo.....	4
Códigos	4
Código de arduino	4
Código HTML del videojuego	9
Estructuras.....	26
Métodos de conexión.....	26
Posibles problemas	28
Soluciones.....	28
Métodos de aseguramiento	31
Evidencias	32
Conclusiones	33
Link de Repositorio de GitHub.....	34
Fotos de la Practica (No tomamos video)	34

Introducción

En la era actual de los sistemas embebidos y la interconectividad, el Internet de las Cosas (IoT) ha transformado la forma en que interactuamos con los entornos de entretenimiento. El presente proyecto aborda el diseño, desarrollo e implementación de un ecosistema tecnológico a escala para la simulación de un **Centro ARCADE Inteligente**. Utilizando como núcleo de control el microcontrolador **ESP32**, el sistema integra tecnologías de comunicación inalámbrica de corto alcance (**RFID/NFC**) y redes locales (**Wi-Fi**) para centralizar la autenticación de usuarios y la gestión de puntuaciones en tiempo real.

El flujo operativo del sistema abarca desde la capa física hasta el backend:

1. **Identificación y Control:** La ESP32 escanea el identificador único (UID) de una tarjeta o llavero inteligente mediante un módulo lector RFID/NFC.
2. **Puente de Red:** Aprovechando su conectividad Wi-Fi, el microcontrolador realiza peticiones HTTP hacia un servidor local para verificar el estado del usuario.
3. **Infraestructura de Datos:** El backend, compuesto por un servidor en **Node.js (JavaScript)** y una base de datos **Microsoft SQL Server** dockerizada en un entorno MacOS, procesa la solicitud. Si el usuario no existe, el sistema aprovisiona de forma automatizada un perfil genérico.
4. **Interacción con el Videojuego:** Una vez validada la identidad, la ESP32 autoriza el inicio de la sesión de juego. Al finalizar la partida, el puntaje obtenido es retornado al microcontrolador, el cual se encarga de enviarlo mediante métodos **POST/PUT** al servidor para actualizar el Score global de forma persistente.

Un aspecto crítico de este proyecto es su enfoque en la **interoperabilidad y estandarización**. La validación final no solo depende del correcto funcionamiento del hardware con su propio videojuego, sino de su capacidad para integrarse de manera agnóstica con los videojuegos desarrollados por los demás equipos. Esto obliga a diseñar interfaces de comunicación (APIs) y protocolos de red sumamente robustos, limpios y bien documentados, simulando un entorno de producción profesional donde diferentes sistemas deben convivir y entenderse bajo las mismas reglas de red.

Desarrollo

Códigos

Código de arduino

```
#include <WiFi.h>
```

```
#include <HTTPClient.h>
```

```
#include <Wire.h>
```

```
#include <Adafruit_PN532.h>
```

```
//
```

```
=====
```

```
// 1. CONFIGURACIÓN I2C — ESP32 WROOM-32
```

```
// SDA → D21 | SCL → D22 (pines I2C nativos de la WROOM-32)
```

```
//
```

```
=====
```

```
Adafruit_PN532 nfc(21, 22); // constructor I2C: (SDA_pin, SCL_pin)
```

```
//
```

```
=====
```

```
// 2. CONFIGURACIÓN DE RED
```

```
//
```

```
=====
```

```
const char* ssid = "TP-Link_3262";
```

```
const char* password = "99428167";
```

```
const String url_base_api = "http://192.168.0.104:3000/api/puntuaciones";
```

```

//
=====

// 3. SETUP

//
=====

void setup(void) {
    Serial.begin(115200);
    while (!Serial) delay(10);

    Serial.println("\n=====");
    Serial.println("  INICIANDO MÓDULO ARCADE — ESP32 WROOM-32  ");
    Serial.println("    (I2C: SDA=D21 SCL=D22)    ");
    Serial.println("=====");

    // Wi-Fi
    WiFi.begin(ssid, password);
    Serial.print("[Wi-Fi] Conectando");
    while (WiFi.status() != WL_CONNECTED) {
        delay(500);
        Serial.print(".");
    }
    Serial.println("\n[Wi-Fi] Conectado!");
    Serial.print("[Wi-Fi] IP asignada: ");
    Serial.println(WiFi.localIP());

    // I2C explícito en pines 21/22 (por si acaso)
    Wire.begin(21, 22);

```

```

// PN532

nfc.begin();

uint32_t versiondata = nfc.getFirmwareVersion();

if (!versiondata) {

    Serial.println("[ERROR] PN532 no encontrado. Verifica:");

    Serial.println("    - Jumpers en modo I2C (SEL0=ON, SEL1=OFF)");

    Serial.println("    - SDA → D21 | SCL → D22");

    Serial.println("    - VCC → 3.3V | GND → GND");

    while (1);

}

Serial.print("[RFID] PN5");

Serial.print((versiondata >> 24) & 0xFF, HEX);

Serial.print(" | Firmware: ");

Serial.print((versiondata >> 16) & 0xFF, DEC);

Serial.print(":");

Serial.println((versiondata >> 8) & 0xFF, DEC);


nfc.setPassiveActivationRetries(0xFF);

nfc.SAMConfig();


Serial.println("[SISTEMA] Listo — esperando tarjeta RFID...");

Serial.println("=====");

}

```

```
//
=====

// 4. LOOP

//
=====

void loop(void) {

    uint8_t uid[7] = {0};

    uint8_t uidLength = 0;

    boolean success = nfc.readPassiveTargetID(
        PN532_MIFARE_ISO14443A, &uid[0], &uidLength
    );

    if (success) {
        if (WiFi.status() != WL_CONNECTED) {
            Serial.println("[ALERTA] Sin Wi-Fi — tarjeta ignorada.");
            return;
        }

        // UID → String hexadecimal en mayúsculas
        String uidCadena = "";

        for (uint8_t i = 0; i < uidLength; i++) {
            if (uid[i] < 0x10) uidCadena += "0";
            uidCadena += String(uid[i], HEX);
        }

        uidCadena.toUpperCase();
    }
}
```

```

Serial.println("\n-----");

Serial.println("[RFID] UID detectado: " + uidCadena);


// GET → avisa al servidor (llena ultimoEscaneo para que el HTML lo lea)
HTTPClient http;

String url_get = url_base_api + "/" + uidCadena;


Serial.println("[HTTP GET] " + url_get);
http.begin(url_get);
int code = http.GET();


if (code == 200) {
    Serial.println("[OK] Servidor respondió: " + http.getString());
    Serial.println("[OK] El juego HTML arrancará en ~1.5 segundos.");
} else {
    Serial.print("[ERROR HTTP] Código: ");
    Serial.println(code);
}


http.end();

Serial.println("-----");


// 6 s de espera para no releer la misma tarjeta
delay(6000);

Serial.println("[SISTEMA] Listo para el siguiente jugador...");
}

```

```
}
```

Código HTML del videojuego

```
<!DOCTYPE html>
```

```
<html lang="es">
```

```
<head>
```

```
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
```

```
  <title>Pac-Man Arcade RFID Edition</title>
```

```
  <style>
```

```
    @import
```

```
url('https://fonts.googleapis.com/css2?family=Press+Start+2P&display=swap');
```

```
    :root{
```

```
      --azul: #0000ff; --amarillo: #ffff00; --rosa: #ffb8ae;
```

```
      --rojo: #ff0000; --cyan: #00ffff; --gris: #555; --fondo: #111;
```

```
    }
```

```
    * { box-sizing: border-box; margin: 0; padding: 0; }
```

```
    body{
```

```
      background: var(--fondo); color: white;
```

```
      font-family: 'Press Start 2P', monospace;
```

```
      display: flex; flex-direction: column;
```

```
      align-items: center; justify-content: flex-start;
```

```
      min-height: 100vh; gap: 10px; padding: 16px 8px;
```

```
    }
```

```
    h1{
```

```
      color: var(--amarillo); font-size: 18px;
```

```
      text-shadow: 0 0 14px var(--rojo), 0 0 30px var(--amarillo);
```

```

    letter-spacing: 3px;
}

#arcade-container {
    position: relative; border: 4px solid var(--azul);
    box-shadow: 0 0 24px var(--azul), inset 0 0 8px #00009a;
    background: black;
}

canvas { display: block; }

/* OVERLAY */

#overlay {
    position: absolute; inset: 0; background: rgba(0,0,0,0.88);
    display: flex; flex-direction: column; align-items: center;
    justify-content: center; text-align: center; gap: 10px;
    z-index: 10; padding: 16px;
}

#overlay h2 { color: var(--cyan); font-size: 13px; line-height: 1.8; text-shadow: 0 0 10px
var(--cyan); }

.blink { animation: blinker 1.2s step-start infinite; color: var(--amarillo); font-size: 11px;
}

@keyframes blinker { 50% { opacity: 0; } }

#overlay p.sub { color: #888; font-size: 8px; max-width: 280px; line-height: 1.8; }

/* PLAYER INFO */

#player-info { display: none; gap: 16px; font-size: 9px; color: white; }

#player-info span { color: var(--amarillo); }

```

```
/* STATUS BAR */

#status-bar { font-size: 8px; color: var(--gris); display: flex; align-items: center; gap: 6px;
}

.dot { display: inline-block; width: 8px; height: 8px; border-radius: 50%; background:
red; }

.dot.ok { background: lime; }


/* DEBUG PANEL */

#debug-panel {

    width: 440px; background: #0a0a0a; border: 1px solid #333;

    padding: 8px; font-size: 8px; font-family: 'Courier New', monospace;
}

#debug-panel .debug-title {

    color: #666; margin-bottom: 6px; font-size: 8px;

    display: flex; justify-content: space-between; align-items: center;
}

#debug-log { color: #0f0; max-height: 110px; overflow-y: auto; line-height: 1.7; }

#debug-log .err { color: #f55; }

#debug-log .ok { color: #5f5; }

#debug-log .warn { color: #fa0; }


/* BOTONES DE DEBUG */

.debug-btns { display: flex; gap: 6px; margin-top: 8px; flex-wrap: wrap; }

.debug-btns button {

    background: #1a1a1a; color: #aaa; border: 1px solid #444;

    padding: 5px 8px; font-size: 7px; font-family: 'Courier New', monospace;

    cursor: pointer; transition: all 0.15s;
}
```

```

}

.debug-btns button:hover { background: #333; color: white; border-color: var(--cyan); }
.debug-btns button.primary { border-color: var(--amarillo); color: var(--amarillo); }

/* IP CONFIG */

#ip-row { display: flex; gap: 6px; align-items: center; margin-top: 6px; }
#ip-row input {
    background: #111; color: #0f0; border: 1px solid #333;
    padding: 4px 6px; font-size: 9px; font-family: 'Courier New', monospace;
    width: 210px; outline: none;
}
#ip-row input:focus { border-color: var(--cyan); }
#ip-row button {
    background: #1a1a1a; color: var(--cyan); border: 1px solid var(--cyan);
    padding: 4px 8px; font-size: 8px; font-family: 'Courier New', monospace; cursor:
pointer;
}

</style>
</head>
<body>

<h1>Ⓢ PAC-MAN ARCADE Ⓢ</h1>

<div id="player-info">
    JUGADOR: <span id="pi-usr">—</span> &nbsp;&nbsp;&nbsp; SALDO: <span id="pi-
score">0</span> pts
</div>

```

```
<div id="arcade-container">
  <div id="overlay">
    <h2>INSERT COIN<br>ALTERNATIVO</h2>
    <p class="blink">◀ PASE SU TARJETA RFID ▶</p>
    <p class="sub">El juego inicia cuando el ESP32 valide tu token.</p>
    <p id="poll-status" style="font-size:8px;color:#555;">Iniciando polling...</p>
  </div>
  <canvas id="pacmanCanvas" width="440" height="440"></canvas>
</div>
```

```
<div id="status-bar">
  <span class="dot" id="dot"></span>
  Servidor: <span id="srv-status">Verificando...</span>
</div>
```

```
<!-- ===== -->
<!--  PANEL DE DEBUG / DIAGNÓSTICO      -->
<!-- ===== -->
```

```
<div id="debug-panel">
  <div class="debug-title">
    <span> ▶ CONSOLA DE DIAGNÓSTICO</span>
    <span id="poll-count" style="color:#555;">polls: 0</span>
  </div>
  <div id="debug-log">Iniciando...</div>
```

```
<div class="debug-btns">

  <button class="primary" onclick="testConexion()">① PING SERVIDOR</button>

  <button onclick="testUltimoMovimiento()">② VER ULTIMO-MOVIMIENTO</button>

  <button onclick="simularRFID()">③ SIMULAR RFID MANUAL</button>

  <button onclick="limpiarLog()" style="color:#555;">LIMPIAR LOG</button>

</div>
```

```
<div id="ip-row">

  <input id="ip-input" type="text" placeholder="http://192.168.0.101:3000" />

  <button onclick="cambiarIP()">APLICAR IP</button>

</div>
```

```
</div>
```

```
<script>
```

```
//
```

```
// CONFIG
```

```
//
```

```
let API_BASE = localStorage.getItem("arcade_api") || "http://192.168.0.101:3000";
```

```
document.getElementById("ip-input").value = API_BASE;
```

```
function cambiarIP() {
```

```
  const val = document.getElementById("ip-input").value.trim().replace(/\/$/, "");
```

```
  if (!val.startsWith("http")) { log("IP inválida — debe empezar con http://", "err"); return; }
```

```
  API_BASE = val;
```

```
  localStorage.setItem("arcade_api", API_BASE);
```

```
log(`IP actualizada → ${API_BASE}`, "ok");  
detenerPolling();  
iniciarPolling();  
}
```

```
//
```

```
// LOGGER VISUAL
```

```
//
```

```
let pollCount = 0;
```

```
function log(msg, tipo = "") {
```

```
    const box = document.getElementById("debug-log");
```

```
    const t = new Date().toLocaleTimeString("es-MX", {hour12:false});
```

```
    const div = document.createElement("div");
```

```
    div.className = tipo;
```

```
    div.textContent = `[${t}] ${msg}`;
```

```
    box.appendChild(div);
```

```
    // Mantener solo últimas 30 líneas
```

```
    while (box.children.length > 30) box.removeChild(box.firstChild);
```

```
    box.scrollTop = box.scrollHeight;
```

```
}
```

```
function limpiarLog() { document.getElementById("debug-log").innerHTML = ""; }
```

```
//
```

```
// ESTADO GLOBAL
```

```
//
```

```
const canvas = document.getElementById("pacmanCanvas");
```

```
const ctx = canvas.getContext("2d");
```

```
const overlay = document.getElementById("overlay");
```

```
let gameInterval = null;
```

```
let score = 0;
```

```
let gameOver = false;
```

```
let jugadorRFID = null;
```

```
let jugadorUSR = "";
```

```
let jugadorSaldoPrev = 0;
```

```
let pollingInterval = null;
```

```
//
```

```
// MAPA
```

```
//
```

```
const mapaOriginal = [
```

```
  [1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1],
```

```
  [1,0,0,0,0,0,0,0,0,0,1,1,0,0,0,0,0,0,0,1],
```

```
  [1,0,1,1,1,0,1,1,1,0,1,1,0,1,1,1,0,1,1,0,1],
```

```
  [1,0,1,1,1,0,1,1,1,0,1,1,0,1,1,1,0,1,1,0,1],
```

```
  [1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1],
```

```
  [1,0,1,1,1,0,1,0,1,1,1,1,1,1,0,1,0,1,1,0,1],
```

```
  [1,0,0,0,0,0,1,0,0,0,1,1,0,0,0,1,0,0,0,0,1],
```

```

[1,1,1,1,1,0,1,1,1,2,1,1,2,1,1,1,0,1,1,1,1,1],
[2,2,2,2,1,0,1,2,2,2,2,2,2,2,1,0,1,2,2,2,2],
[1,1,1,1,1,0,1,2,1,1,2,2,1,1,2,1,0,1,1,1,1,1],
[2,2,2,2,2,0,2,2,1,2,2,2,2,1,2,2,0,2,2,2,2,2],
[1,1,1,1,1,0,1,2,1,1,1,1,1,1,2,1,0,1,1,1,1,1],
[2,2,2,2,1,0,1,2,2,2,2,2,2,2,1,0,1,2,2,2,2],
[1,1,1,1,1,0,1,2,1,1,1,1,1,1,2,1,0,1,1,1,1,1],
[1,0,0,0,0,0,0,0,0,0,1,1,0,0,0,0,0,0,0,0,0,1],
[1,0,1,1,1,0,1,1,1,0,1,1,0,1,1,0,1,1,1,0,1],
[1,0,0,0,1,0,0,0,0,0,2,2,0,0,0,0,0,1,0,0,0,1],
[1,1,1,0,1,0,1,0,1,1,1,1,1,1,0,1,0,1,0,1,1,1],
[1,0,0,0,0,0,1,0,0,0,1,1,0,0,0,1,0,0,0,0,0,1],
[1,0,1,1,1,1,1,1,1,0,1,1,0,1,1,1,1,1,1,0,1],
[1,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,0,1],
[1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1,1]
];

let map = mapaOriginal.map(r => [...r]);

const tileSize = 20;

let pacman = { x:1, y:1, dirX:0, dirY:0, nextDirX:0, nextDirY:0 };

let ghost = { x:10, y:10, dirX:1, dirY:0, color:"red" };

//


---




---


// POLLING
//


---




---



function iniciarPolling() {

```

```

log(` Polling iniciado → ${API_BASE}/api/ultimo-movimiento`, "ok");

setSrvStatus("Polling activo...", true);


pollingInterval = setInterval(async () => {

  pollCount++;

  document.getElementById("poll-count").textContent = ` polls: ${pollCount}`;

  try{

    const res = await fetch(`${API_BASE}/api/ultimo-movimiento`, { cache: "no-store" });

    if (!res.ok) {

      log(` HTTP ${res.status} en ultimo-movimiento`, "err");

      setSrvStatus(` Error HTTP ${res.status}`, false);

      return;

    }

    const data = await res.json();

    const id = data.id_rfid || "";

    // Mostrar la respuesta raw cada 10 polls para no saturar

    if (pollCount % 10 === 0) {

      log(` Respuesta: ${JSON.stringify(data)}`, "");

    }

    if (id && id !== "-----" && id !== "") {

      log(` ✓ RFID detectado: ${id} | usr: ${data usr} | score: ${data.score}`, "ok");

      detenerPolling();

```

```

        jugadorRFID    = id;

        jugadorUSR     = data.usr || `User_${id.substring(0,4)}`;

        jugadorSaldoPrev = data.score || 0;

        arrancarJuego();

    }

} catch (e) {

    log(`Fetch falló: ${e.message}`, "err");

    setSrvStatus("Sin respuesta", false);

}

}, 1500);

}

```

```

function detenerPolling() {

    clearInterval(pollingInterval);

    pollingInterval = null;

    log("Polling detenido.", "warn");

}

```

```
//
```

```
// FUNCIONES DE DEBUG (botones)
```

```
//
```

```

async function testConexion() {

    log(`Probando ping a ${API_BASE}/api/ultimo-movimiento ...`, "warn");

    try {

        const r = await fetch(`${API_BASE}/api/ultimo-movimiento`, { cache: "no-store" });
    }

```

```

    const d = await r.json();

    log(`PING OK ✓ → ${JSON.stringify(d)}`, "ok");

    setSrvStatus("Conectado", true);
} catch(e) {

    log(`PING FALLÓ X → ${e.message}`, "err");

    log("Posibles causas: IP incorrecta, CORS bloqueado, servidor caído.", "err");

    setSrvStatus("Sin conexión", false);

}
}

```

```

async function testUltimoMovimiento() {

    log("Consultando /api/ultimo-movimiento ...", "warn");

    try {

        const r = await fetch(`${API_BASE}/api/ultimo-movimiento`, { cache: "no-store" });

        const d = await r.json();

        log(`Respuesta completa: ${JSON.stringify(d)}`, d.id_rfid !== "-----" ? "ok" : "warn");

    } catch(e) {

        log(`Error: ${e.message}`, "err");

    }

}

```

```

function simularRFID() {

    log("Simulando escaneo RFID manual (RFID_TEST_001)...", "warn");

    detenerPolling();

    jugadorRFID    = "RFID_TEST_001";

    jugadorUSR     = "TestPlayer";

```

```
jugadorSaldoPrev = 500;

arrancarJuego();

}
```

```
//
```

```
// JUEGO
```

```
//
```

```
function arrancarJuego() {
    map = mapaOriginal.map(r => [...r]);
    score = 0; gameOver = false;
    pacman = { x:1, y:1, dirX:0, dirY:0, nextDirX:0, nextDirY:0 };
    ghost = { x:10, y:10, dirX:1, dirY:0, color:"red" };
    document.getElementById("pi-usr").textContent = jugadorUSR;
    document.getElementById("pi-score").textContent = jugadorSaldoPrev;
    document.getElementById("player-info").style.display = "flex";
    overlay.style.display = "none";
    if (gameInterval) clearInterval(gameInterval);
    gameInterval = setInterval(update, 150);
    log(` Juego iniciado para ${jugadorUSR} (saldo previo: ${jugadorSaldoPrev})`, "ok");
}
```

```
function volverAEspera() {
    clearInterval(gameInterval);
    gameInterval = null;
    jugadorRFID = null;
}
```

```
document.getElementById("player-info").style.display = "none";
overlay.style.display = "flex";
document.getElementById("poll-status").textContent = "Esperando nueva tarjeta...";
pollCount = 0;
log("—— Volviendo a pantalla de espera ——", "warn");
iniciarPolling();
}
```

```
window.addEventListener("keydown", e => {
  const dirs = {
    ArrowUp:[0,-1], ArrowDown:[0,1], ArrowLeft:[-1,0], ArrowRight:[1,0],
    w:[0,-1], s:[0,1], a:[-1,0], d:[1,0]
  };
  const d = dirs[e.key];
  if (d) { pacman.nextDirX = d[0]; pacman.nextDirY = d[1]; }
});
```

```
function update() {
  if (gameOver) return;
  if (map[pacman.y + pacman.nextDirY]?.[pacman.x + pacman.nextDirX] !== 1) {
    pacman.dirX = pacman.nextDirX; pacman.dirY = pacman.nextDirY;
  }
  if (map[pacman.y + pacman.dirY]?.[pacman.x + pacman.dirX] !== 1) {
    pacman.x += pacman.dirX; pacman.y += pacman.dirY;
  }
  if (map[pacman.y][pacman.x] === 0) { map[pacman.y][pacman.x] = 2; score += 10; }
```

```

const dirs = [{x:1,y:0},{x:-1,y:0},{x:0,y:1},{x:0,y:-1}];

const validos = dirs.filter(d => map[ghost.y+d.y]?.[ghost.x+d.x] !== 1);

if (validos.length > 0 && (Math.random() < 0.3 ||
map[ghost.y+ghost.dirY]?.[ghost.x+ghost.dirX] === 1)) {

    const e = validos[Math.floor(Math.random() * validos.length)];

    ghost.dirX = e.x; ghost.dirY = e.y;

}

ghost.x += ghost.dirX; ghost.y += ghost.dirY;


if (pacman.x === ghost.x && pacman.y === ghost.y) {

    gameOver = true;

    clearInterval(gameInterval);

    log(`Game Over — score: ${score} pts`, "warn");

    enviarPuntuacion();

    setTimeout(() => {

        alert(`¡GAME OVER!\nPuntuación: ${score} pts\nSaldo previo: ${jugadorSaldoPrev}
pts\nNuevo total enviado al servidor.`);

        volverAEspera();

    }, 80);

}

draw();

}

```

```

async function enviarPuntuacion() {

    if (!jugadorRFID || jugadorRFID === "RFID_TEST_001") {

        log("Score de prueba — no se envía al servidor.", "warn"); return;
    }
}

```

```

    }

    const body = { id_rfid: jugadorRFID, usr: jugadorUSR, name_disp: "Arcade_HTML5", score
};

    log(` POST /api/puntuaciones → ${JSON.stringify(body)} `, "warn");

    try {

        const res = await fetch(` ${API_BASE}/api/puntuaciones `, {

            method: "POST", headers: { "Content-Type": "application/json" },

            body: JSON.stringify(body)

        });

        const data = await res.json();

        log(` POST OK ✓ → total_score: ${data.total_score} `, "ok");

    } catch (e) {

        log(` POST falló: ${e.message} `, "err");

    }

}

```

```
//
```

```
// RENDER
```

```
//
```

```

function draw() {

    ctx.clearRect(0,0,canvas.width,canvas.height);

    ctx.fillStyle = "black"; ctx.fillRect(0,0,canvas.width,canvas.height);

    for (let r=0;r<map.length;r++) for (let c=0;c<map[r].length;c++) {

        if (map[r][c]===1) {

            ctx.fillStyle="#0000cc"; ctx.fillRect(c*tileSize,r*tileSize,tileSize,tileSize);

```

```

        ctx.strokeStyle="#4444ff"; ctx.lineWidth=1;

        ctx.strokeRect(c*tileSize+1,r*tileSize+1,tileSize-2,tileSize-2);

    } else if (map[r][c]===0) {

        ctx.fillStyle="#ffb8ae"; ctx.beginPath();

        ctx.arc(c*tileSize+tileSize/2,r*tileSize+tileSize/2,3,0,Math.PI*2); ctx.fill();

    }

}

const rad=tileSize/2, cx=pacman.x*tileSize+rad, cy=pacman.y*tileSize+rad;

ctx.fillStyle="#ffff00"; ctx.beginPath();

ctx.arc(cx,cy,rad-1,0.25*Math.PI,1.75*Math.PI); ctx.lineTo(cx,cy); ctx.fill();


const gx=ghost.x*tileSize, gy=ghost.y*tileSize;

ctx.fillStyle=ghost.color; ctx.beginPath();

ctx.arc(gx+rad,gy+rad,rad-1,Math.PI,0,false);

ctx.lineTo(gx+tileSize-1,gy+tileSize); ctx.lineTo(gx+1,gy+tileSize); ctx.fill();

ctx.fillStyle="white"; ctx.fillRect(gx+4,gy+4,4,4); ctx.fillRect(gx+12,gy+4,4,4);

ctx.fillStyle="blue"; ctx.fillRect(gx+5,gy+5,2,2); ctx.fillRect(gx+13,gy+5,2,2);


ctx.fillStyle="white"; ctx.font="bold 13px 'Courier New'";

ctx.fillText(` SCORE: ${score}` ,10,14);

}

function setSrvStatus(msg,ok) {

    document.getElementById("srv-status").textContent = msg;

    document.getElementById("dot").className = "dot" + (ok?" ok":"" );

}

```

```
draw();

iniciarPolling();

</script>

</body>

</html>
```

Estructuras

1. **Disparo por Evento:** Al suscitarse la colisión de Pac-Man con un fantasma (gameOver = true), el cliente web interrumpe el ciclo de refresco gráfico.
2. **Emisión del Estado:** El navegador utiliza el canal activo de WebSockets para emitir hacia el servidor el evento "guardar_puntuacion".

3. Estructura del JSON (Salida del Juego):

JSON

```
{
  "id_rfid": "A1B2C3D4",
  "score": 480,
  "name_disp": "JUGADOR ARCADE"
}
```

4. **Persistencia Relacional:** El backend en Node.js captura la trama del WebSocket e inyecta las variables en una consulta parametrizada que ejecuta un INSERT directo sobre la tabla puntuaciones dentro del contenedor Docker de la base de datos arcade_db.

Métodos de conexión

Para la captura de las credenciales físicas de los usuarios en la cabina arcade, se implementó el módulo lector **PN532 v3** acoplado al módulo de desarrollo embebido **NodeMCU ESP32 (V1)**.

A diferencia de las configuraciones tradicionales basadas en buses SPI (que saturan el diseño con exceso de cableado), este sistema se optimizó utilizando el bus de comunicación **I2C (Inter-Integrated Circuit)**. Esta arquitectura reduce la interfaz a una topología de bus serie de solo dos líneas de señal, minimizando el ruido electromagnético y optimizando el espacio físico.

Configuración del Bloque de Conmutación (DIP Switches)

Para forzar al chip controlador del PN532 a conmutar sus registros internos hacia la interfaz física del bus I2C (ubicada en el extremo izquierdo del módulo), se realizó el ajuste de los microinterruptores físicos bajo el siguiente esquema lógico grabado en el PCB:

- **Interruptor 1 (SEL0):** ON (Hacia arriba / Nivel lógico alto)
- **Interruptor 2 (SEL1):** OFF (Hacia abajo / Nivel lógico bajo)

Mapeo de Pines Físicos

La interfaz de hardware se interconectó directamente a los pines maestros de la arquitectura del ESP32 DevKit V1:

Línea de Conexión (PN532 - Lado Izquierdo)	Pin Físico en la Placa ESP32	Tipo de Señal / Función	Descripción Técnica
GND	GND	Referencia	Tierra común del circuito para evitar la flotación de voltajes.
VCC	VIN	Alimentación	Entrada de voltaje directo (5V) desde el bus USB para asegurar estabilidad en la antena de radiofrecuencia (RF).
SDA	D21 (GPIO 21)	Serial Data	Línea de transmisión bidireccional de datos del bus I2C nativo.

Línea de Conexión (PN532 - Lado Izquierdo)	Pin Físico en la Placa ESP32	Tipo de Señal / Función	Descripción Técnica
SCL	D22 (GPIO 22)	Serial Clock	Línea de sincronización de reloj maestro generada por el ESP32.

Posibles problemas

Problema 1: Excepción Crítica "No se encontró el módulo PN532" al arrancar el ESP32

- **Causa Raíz:** Falsa conmutación de los registros lógicos debido a una mala posición en el bloque de switches físicos, o caída de tensión en el pin de alimentación al usar 3.3V.
- **Solución Aplicada:** Validar físicamente que la combinación sea 1:ON y 2:OFF. Migrar el puente de alimentación del pin de 3.3V al pin **VIN (5V)** para compensar las caídas bruscas de corriente cuando la bobina de la antena de inducción RF se activa.

Problema 2: Desconexión intermitente de los clientes WebSocket durante la ejecución del juego

- **Causa Raíz:** Pérdida de paquetes de control (Ping/Pong timeouts) causados por saturación en la red local inalámbrica o por la velocidad de refresco del bucle principal del navegador.
- **Solución Aplicada:** Aislar las peticiones desacoplando las llamadas del ciclo de pintado gráfico del Canvas mediante un intervalo estricto y asíncrono de 150ms. Asimismo, configurar propiedades de reconexión automática (reconnection: true) en el cliente de Socket.io.

Soluciones

PROBLEMA 1: Excepción Crítica "No se encontró el módulo PN532" al arrancar el microcontrolador ESP32

- **Descripción del Síntoma:** Al inicializar el sistema o tras un reinicio de la placa de desarrollo, el monitor serie arroja de forma persistente el error de interrupción en el bucle `setup()` deteniendo la ejecución general del programa. El firmware no logra establecer el apretón de manos (*handshake*) con el bus de periféricos.
- **Análisis de Causa Raíz (RCA):**
 1. **Falsa conmutación de registros lógicos:** El chip controlador del módulo PN532 cuenta con pines selectores internos configurables a través de interruptores integrados (*DIP Switches*). Si la combinación binaria es errónea, el hardware asume líneas de comunicación SPI o UART en lugar de abrir los registros esclavos para el protocolo I2C.
 2. **Inestabilidad de Voltaje por Inducción Electromagnética:** Al alimentar el lector con la salida lineal regulada de 3.3V del ESP32, el módulo opera al límite de su umbral crítico. En el instante exacto en que se inicializa la antena interna de radiofrecuencia (RF) de 13.56 MHz, se genera un pico de demanda de corriente (efecto de carga inductiva) que desploma el voltaje por debajo de los límites lógicos, provocando el colapso del transceptor y congelando el canal I2C.
- **Soluciones Aplicadas e Implementación Técnica:**
 1. **Reconfiguración de la Matriz del Decodificador:** Se validó e intervino el bloque de microinterruptores lógicos en el PCB del PN532, ajustando físicamente la conmutación a la posición binaria exacta para habilitar el direccionamiento por bus I2C: **Canal 1 en ON y Canal 2 en OFF.**
 2. **Redireccionamiento de Potencia hacia el Bus Maestro:** Se eliminó el puente de la línea de 3.3V y se migró la alimentación del pin VCC del lector hacia el pin **VIN** del NodeMCU ESP32. El pin VIN suministra directamente los 5V regulados provenientes del puerto USB de la computadora. Esto dota a la bobina magnética del lector de la tolerancia de corriente necesaria para activar el campo de acoplamiento sin inducir caídas de tensión bruscas en las líneas lógicas seriales (D21/SDA y D22/SCL).

PROBLEMA 2: Desconexión intermitente de los clientes WebSocket (Socket.io) durante la ejecución activa del juego

- **Descripción del Síntoma:** Tras iniciar la partida de Pac-Man, el estado del WebSocket pasa aleatoriamente a "Desconectado" y el juego se congela o retorna a la pantalla de bloqueo `#overlay`. No existe consistencia en la retención del enlace asíncrono.
- **Análisis de Causa Raíz (RCA):**

1. **Saturación y Bloqueo del Hilo de Eventos (*Event Loop*):** El lienzo de renderizado gráfico de HTML5 (Canvas) refresca los elementos visuales a velocidades muy altas. Al acoplar o condicionar las funciones de red (lecturas, estados y comprobaciones del servidor) directamente dentro del ciclo de pintado nativo de los gráficos, se satura el hilo único de ejecución de JavaScript.
2. **Timeouts por Pérdida de Latidos (*Ping/Pong*):** Al no poder procesar a tiempo los paquetes de control debido a la carga gráfica, el motor de Socket.io asume que el cliente ha muerto por inactividad tras exceder el tiempo de espera por defecto, cerrando el socket de forma unilateral por razones de tolerancia a fallos.

- **Soluciones Aplicadas e Implementación Técnica:**

1. **Desacoplamiento Estricto de la Arquitectura de Red:** Se rediseñó la lógica del cliente web separando completamente el ciclo de ejecución gráfica del Canvas de la capa de comunicación de WebSockets. El motor del juego fue encapsulado para operar de forma asíncrona dentro de un temporizador de intervalo periódico (setInterval) controlado estrictamente a **150ms**. Esto libera ciclos de CPU críticos, permitiendo que el navegador procese los paquetes de red en segundo plano sin interrumpir ni retrasar los hilos gráficos.
2. **Inyección de Políticas de Resiliencia en el Cliente:** Se modificó la instanciación de la conexión del lado de la interfaz web configurando explícitamente parámetros avanzados de persistencia y tolerancia a fallos en el constructor de Socket.io:

JavaScript

```
const socket = io("http://localhost:3000", {  
  reconnection: true,      // Habilita reconexiones automáticas  
  reconnectionAttempts: Infinity, // Intentos ilimitados si cae el servidor  
  reconnectionDelay: 1000,  // Reintento cada 1 segundo exacto  
  timeout: 20000           // Aumenta el umbral de desconexión por ping  
});
```

Con esta directiva, ante cualquier micro-corte o fluctuación física en la red inalámbrica, el cliente web se reengancha al socket de manera transparente y en milisegundos, salvaguardando el progreso, la puntuación actual de la sesión y previniendo la interrupción abrupta de la experiencia arcade.

Métodos de aseguramiento

Para garantizar la robustez del entorno frente a entornos de producción reales, se implementaron los siguientes mecanismos de control:

1. **Parametrización Estricta de Consultas (Prevención de Inyecciones SQL):** Toda interacción con la base de datos a través de Node.js se gestiona mediante objetos `sql.input()`. Las variables de los clientes (`id_rfid`, `score`) jamás se concatenan directamente en cadenas de texto nativas, anulando cualquier vector de ataque de inyección de código SQL malicioso.
2. **Filtrado de Entradas con Expresiones Regulares (Sanitización de Datos):** El UID capturado es forzado a transformarse a una estructura string controlada en mayúsculas mediante métodos nativos tanto en el ESP32 como en Node.js, bloqueando caracteres especiales ajenos al alfabeto hexadecimal antes de su procesamiento.
3. **Control de Concurrencia de Lectura (Anti-Bounce por Software):** Para evitar escrituras duplicadas o transacciones masivas accidentales en SQL Server debido a la permanencia prolongada de una tarjeta sobre el lector físico, se insertó un temporizador de bloqueo por hardware (`delay(2000)`) en el firmware del ESP32 que inhibe la antena inmediatamente después de una lectura exitosa.

Evidencias



The screenshot shows a SQL Server Enterprise Manager interface. The query editor displays the following SQL statement:

```
SELECT TOP (100) [ID]
FROM [arcade_db].[dbo].[tblGame]
WHERE [ID] < 1000
```

The query results are displayed in a table with the following columns: ID, NAME_DISP, USR, SCORE, LAST_GAME, and ID_RFID. The results are sorted by ID in descending order.

ID	NAME_DISP	USR	SCORE	LAST_GAME	ID_RFID
1027	Arcade_HTML5	User_7DF2	610	2026-06-04 19:41:08.573	7DF21707
1028	DEAL OR NOT DEAL	Fulanotedetel	394	2026-06-04 19:34:45.713	ed710207
1029	Arcade_Naves_Jorge	User_07F1	100	2026-06-04 19:38:27.367	07F17764
1030	cardCad	cardCad	20	2026-06-04 19:35:13.597	0D101207
1031	User_0EB4	User_0EB4	3450	2026-06-04 19:40:16.717	0EB41407
1032	Snake	NELLY	4810	2026-06-04 19:41:19.840	02732807
1033	DEAL OR NOT DEAL	Fulanotedetel	874	2026-06-04 19:40:44.100	3EEE1607
1034	Jugador	Jugador	50	2026-06-04 19:39:09.843	01372707
1035	Inalambricas	Jugador	100	2026-06-04 19:37:34.860	00935021
1036	Inalambricas	Jugador	0	2026-06-04 19:38:00.723	00135E72



The screenshot shows a SQL Server Enterprise Manager interface. The query editor displays the following SQL statement:

```
SELECT TOP (1000) [ID]
FROM [arcade_db].[dbo].[tblGame]
WHERE [ID] < 1000
```

The query results are displayed in a table with the following columns: ID, NAME_DISP, USR, SCORE, LAST_GAME, and ID_RFID. The results are sorted by ID in descending order.

ID	NAME_DISP	USR	SCORE	LAST_GAME	ID_RFID
1027	Arcade_HTML5	User_7DF2	610	2026-06-04 19:41:08.573	7DF21707
1028	DEAL OR NOT DEAL	Fulanotedetel	394	2026-06-04 19:34:45.713	ed710207
1029	Arcade_Naves_Jorge	User_07F1	100	2026-06-04 19:38:27.367	07F17764
1030	cardCad	cardCad	20	2026-06-04 19:35:13.597	0D101207
1031	User_0EB4	User_0EB4	3450	2026-06-04 19:40:16.717	0EB41407
1032	Snake	NELLY	4810	2026-06-04 19:41:19.840	02732807
1033	DEAL OR NOT DEAL	Fulanotedetel	874	2026-06-04 19:40:44.100	3EEE1607
1034	Jugador	Jugador	50	2026-06-04 19:39:09.843	01372707
1035	Inalambricas	Jugador	100	2026-06-04 19:37:34.860	00935021
1036	Inalambricas	Jugador	0	2026-06-04 19:38:00.723	00135E72
1037	Inalambricas	Jugador	0	2026-06-04 19:38:11.697	00EC2283
1038	Inalambricas	Jugador	160	2026-06-04 19:51:10.953	2E401307
1039	Arcade_Naves_Jorge	User_E740	500	2026-06-04 19:52:13.213	E7401507
1040	Inalambricas	Jugador	100	2026-06-04 19:44:31.933	97D7CA06
1041	cardCad	cardCad	60	2026-06-04 19:41:15.970	2E
1042	User_ID2C	User_ID2C	1960	2026-06-04 19:47:52.893	1D2C6C06
1043	Snake	NELLY	30	2026-06-04 19:53:49.963	90972407
1044	DEAL OR NOT DEAL	Fulanotedetel	890	2026-06-04 19:53:34.950	3D3CFD06
1045	Jugador	Jugador	20	2026-06-04 19:44:11.553	C9
1046	Inalambricas	Jugador	120	2026-06-04 19:53:43.018	63500D04
1047	Inalambricas	Jugador	10	2026-06-04 19:51:53.757	01971507
1048	Inalambricas	Jugador	0	2026-06-04 19:52:08.670	93D61806

Conclusiones

Dávila Jasso Axhel Giovanni

Con base a esta práctica final, logré consolidar de manera integral los conocimientos adquiridos a lo largo del curso, enfrentándome al reto de diseñar un ecosistema IoT completo desde cero. El desarrollo de este centro ARCADE me permitió comprender la importancia de la interoperabilidad en la ingeniería actual; no bastaba con que mi código o mi hardware funcionaran de forma aislada, sino que debían adaptarse de manera agnóstica a los sistemas de los demás equipos. La experiencia de desplegar una base de datos SQL Server en contenedores Docker sobre MacOS, coordinada con la lógica de la ESP32 y un servidor en Node.js, me brindó una visión real sobre la arquitectura de software y hardware. Al final, este proyecto me demostró que, como futuro ingeniero, el verdadero valor técnico radica en la capacidad de construir soluciones robustas, modulares y capaces de integrarse en entornos de red colaborativos y heterogéneos.

Flores Dueñas Elvia Yuridia

Con base a esta práctica final, puse a prueba mis habilidades técnicas e ingenieriles al integrar disciplinas esenciales como los sistemas embebidos, el desarrollo de software y la administración de servidores. La simulación de este centro ARCADE representó un desafío significativo, especialmente al estandarizar los métodos de comunicación HTTP para asegurar que cualquier videojuego pudiera interactuar con la ESP32 y el backend de manera fluida. El proceso de configurar el contenedor de Docker para Microsoft SQL Server y desarrollar la lógica en JavaScript me permitió dominar el flujo de datos de extremo a extremo (end-to-end). Esta experiencia me deja una gran satisfacción y me confirma que la clave de la ingeniería moderna no solo es resolver problemas de forma individual, sino diseñar arquitecturas limpias, escalables y seguras que faciliten el trabajo en equipo y la compatibilidad entre plataformas.

Link de Repositorio de GitHub

<https://github.com/AXHEL1905/-Evaluativo-FINAL>

Fotos de la Practica (No tomamos video)

